01-MultiAgent 보충 자료

# A2A 멀티 에이전트 샘플 코드 심층 분석

Incident Response 샘플로 보는 Runtime · Identity · Gateway · Memory
Building Advanced Agentic Systems on AWS

**Google ADK**
호스트 에이전트

**Strands Agents**
모니터링 에이전트

**OpenAI Agents SDK**
웹 검색 에이전트

# 이 자료의 구성

A

**교안 ↔ 코드 매핑**
교안 슬라이드를 진행하며 화면에 띄울 코드 위치 13곳

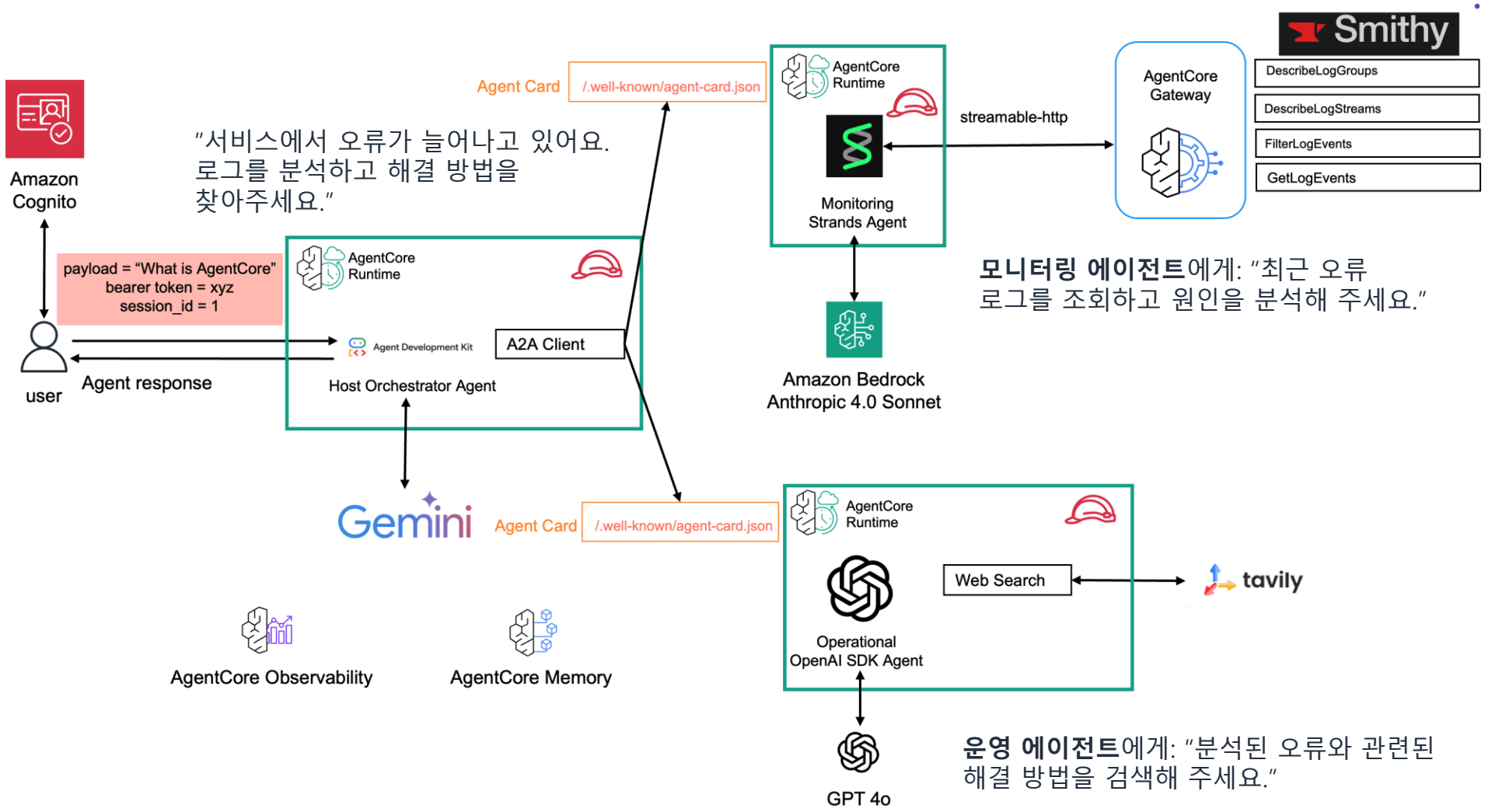
B

**추가적인 기술 내용**
Runtime 계약, 헤더 전파, 인증 체인, ADK 위임, A2A 실행기, Gateway, 메모리 전략, 세션 격리, 배포

C

**코드에서 찾은 함정**
보안, 프롬프트, 복원력 문제 6가지

# 샘플 아키텍처: 런타임 3개, 프레임워크 3개



# A

# 교안 ↔ 코드 매핑

교안을 진행하면서 반드시 화면에 띄울 코드 위치

# 통신 패턴: 꼭 짚을 코드 7곳

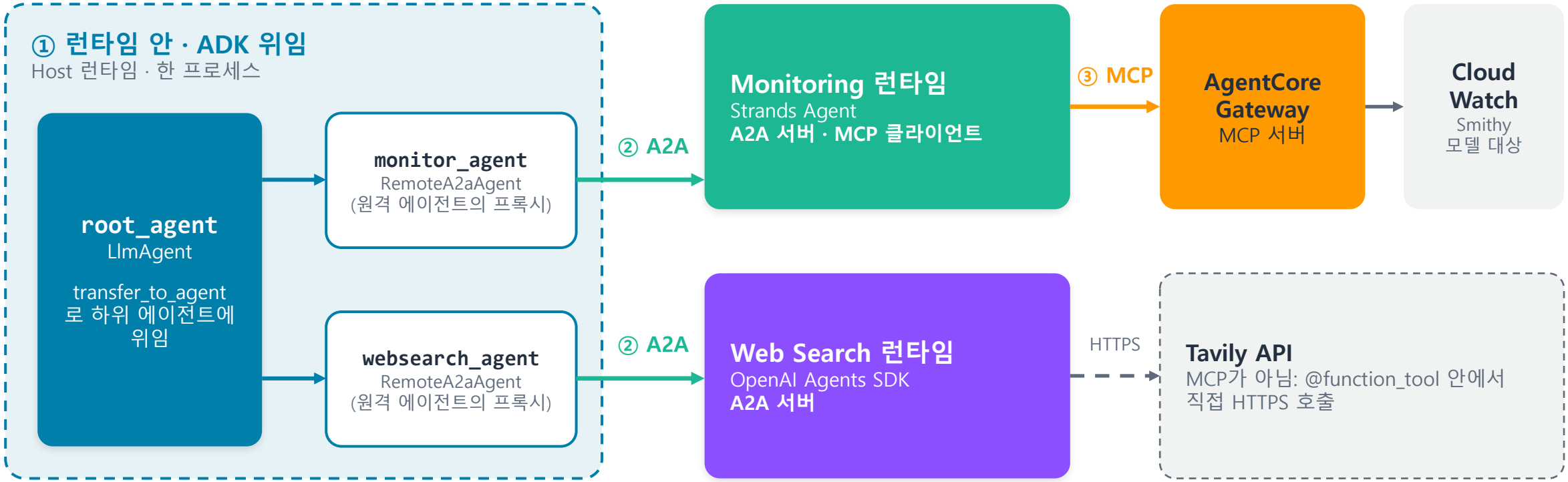
교안 #15–25

| 교안 | 개념 | 코드 위치 | 짚을 포인트 |
|---|---|---|---|
| **#15** | 패턴 혼합 | `host_adk_agent/agent.py:151` | 런타임 안은 ADK 위임, 런타임 사이는 A2A, 도구는 MCP |
| **#17** | 도구로서의 에이전트 | `host_adk_agent/prompt/__init__.py` | description과 위임 규칙이 라우팅 기준 (monitor → websearch → 합성) |
| **#18** | A2A 클라이언트 | `host_adk_agent/agent.py:122` | RemoteA2aAgent — 원격 에이전트가 로컬 sub_agent와 같은 자리 |
| **#18** | A2A 서버 · 카드 | `monitoring_strands_agent/main.py:37, 83` | AgentCard → /.well-known/agent-card.json 자동 게시 |
| **#18** | 태스크 · 아티팩트 | `monitoring_strands_agent/agent_executor.py:144, 98` | new_task → working → add_artifact("agent_response") → complete |
| **#24** | 인터페이스 계약 | `monitoring_strands_agent/agent_executor.py:130` | 입출력 모드, skills, version, JSON-RPC 오류 (InvalidParamsError) |
| **#25** | 보안 | `host_adk_agent/agent.py:40 · cloudformation/*.yaml` | Runtime의 CustomJWTAuthorizer + @requires_access_token(M2M) |

# 한 시스템, 세 개의 통신 계층

host_adk_agent/agent.py:151

교안 #15




**① 런타임 안 · ADK 위임**
**범위** 같은 프로세스 · 같은 메모리
**수단** sub_agents + transfer_to_agent
교안 #16 프레임워크 계층 · `agent.py:151-156`

**② 런타임 사이 · A2A**
**범위** 독립 런타임 (세션별 microVM)
**수단** JSON-RPC over HTTPS · 카드 · M2M JWT
교안 #18 에이전트 간 · `agent.py:122, 139`

**③ 에이전트 ↔ 도구 · MCP**
**범위** 에이전트와 외부 도구 사이
**수단** Gateway (Streamable HTTP) · 도구 목록
교안 #15 MCP · `utils.py:25 (monitoring)`

# 라우팅 기준: description과 위임 규칙

교안 #17

### 입력 ① SYSTEM_PROMPT

```
# host_adk_agent/prompt/__init__.py
**Delegation Rules:**
- monitor_agent: metrics, logs, alarms
- websearch_agent: guides, docs, fixes

**Orchestration Strategy:**
1. First  → monitor_agent
2. Then   → websearch_agent (+context)
3. Finally → synthesize
```

### 입력 ② sub_agent description

**monitor_agent**
"Agent that handles monitoring tasks."
**websearch_agent**
"Web search agent for finding AWS …"

사용자 질문
**"EC2 CPU가 높아"**

**root_agent**
LLM이 라우팅 판단

transfer_to_agent(
agent_name)

**1** **monitor_agent**
**현재 상태: 지표 · 로그 · 알람 수집**
"Check current CPU metrics for EC2,
recent spikes, and related alarms"

**2** **websearch_agent**
**해결책: 지표를 맥락으로 붙여 검색**
"Find EC2 high CPU troubleshooting
steps and common causes" + 1단계 지표

**3** **root_agent 합성**
**최종 응답: 우선순위 있는 조치 목록**
지표 + 조치 단계, 수치 · 타임스탬프 인용

**주의** description이 한 줄이라 판단은 대부분 프롬프트 규칙이 좌우합니다. 순서(1 → 2 → 3)는 프롬프트의 지시일 뿐 코드로 보장되지 않습니다 → 파트 B의 ADK sub_agents 슬라이드

# A2A 클라이언트: 원격 에이전트가 sub_agent 자리에

교안 #18

**`root_agent.sub_agents = [ monitor_agent, websearch_agent ]`**
ADK가 보는 것은 BaseAgent 하나: name · description · run_async() → Event

### 로컬 sub_agent였다면
예: `Agent(model=..., tools=[...])`

- 같은 프로세스 · 같은 이벤트 루프
- 모델 호출 + 도구 실행
- 결과를 Event로 반환

### RemoteA2aAgent · 원격 에이전트의 프록시
host_adk_agent/agent.py:122

1. **카드 조회** `_ensure_resolved() → /.well-known/agent-card.json`
2. **클라이언트 생성** `a2a_client_factory → M2M 토큰 + 세션 · 액터 헤더`
3. **A2A 호출** `JSON-RPC message/send → 원격 런타임`
4. **응답 변환** `원격의 Task · Artifact → ADK Event`

```
monitor_agent = RemoteA2aAgent(
    name="monitor_agent", description="Agent that handles monitoring tasks.",
    agent_card=monitor_agent_card_url,              # URL만 넘김
    a2a_client_factory=_create_client_factory(...)) # 인증 · 헤더
```

### 같은 자리 = 교체 가능
원격 ↔ 로컬을 바꿔도 root_agent 코드는 그대로. 지연 · 실패 · 인증은 프록시가 흡수

# A2A 서버: 카드를 만들면 경로가 생긴다

교안 #18

A2AStarletteApplication은 **A2A 통신에 필요한 웹 경로와 요청 처리를 구성해 주는 SDK 구성 요소**

```python
# main.py:37
agent_card = AgentCard(
  name="Monitoring Agent",
  url=runtime_url,   # AGENTCORE_RUNTIME_URL
  version="0.3.0",
  defaultInputModes=["text/plain"],
  defaultOutputModes=["text/plain"],
  capabilities=AgentCapabilities(streaming=True),
  skills=[AgentSkill(...), ...])   # 5개

# main.py:83, 86
server = A2AStarletteApplication(
  agent_card=agent_card,
  http_handler=request_handler)
app = server.build()
```

**A2AStarletteApplication**

**agent_card**
→ 카드 경로 자동 생성

**http_handler**
DefaultRequestHandler
├ MonitoringAgentExecutor
└ InMemoryTaskStore

**server.build() → Starlette 앱**

**GET /.well-known/agent-card.json** — 자동
카드 객체를 JSON으로 반환

**POST /** — 자동
JSON-RPC: message/send · tasks/get · tasks/cancel
→ 핸들러 → 실행기

**GET /ping** — 직접 추가
헬스 체크 (main.py:89)

## AgentCore Runtime을 거칠 때의 경로

**클라이언트 (호스트)**
.../runtimes/{ARN}/invocations
/.well-known/agent-card.json

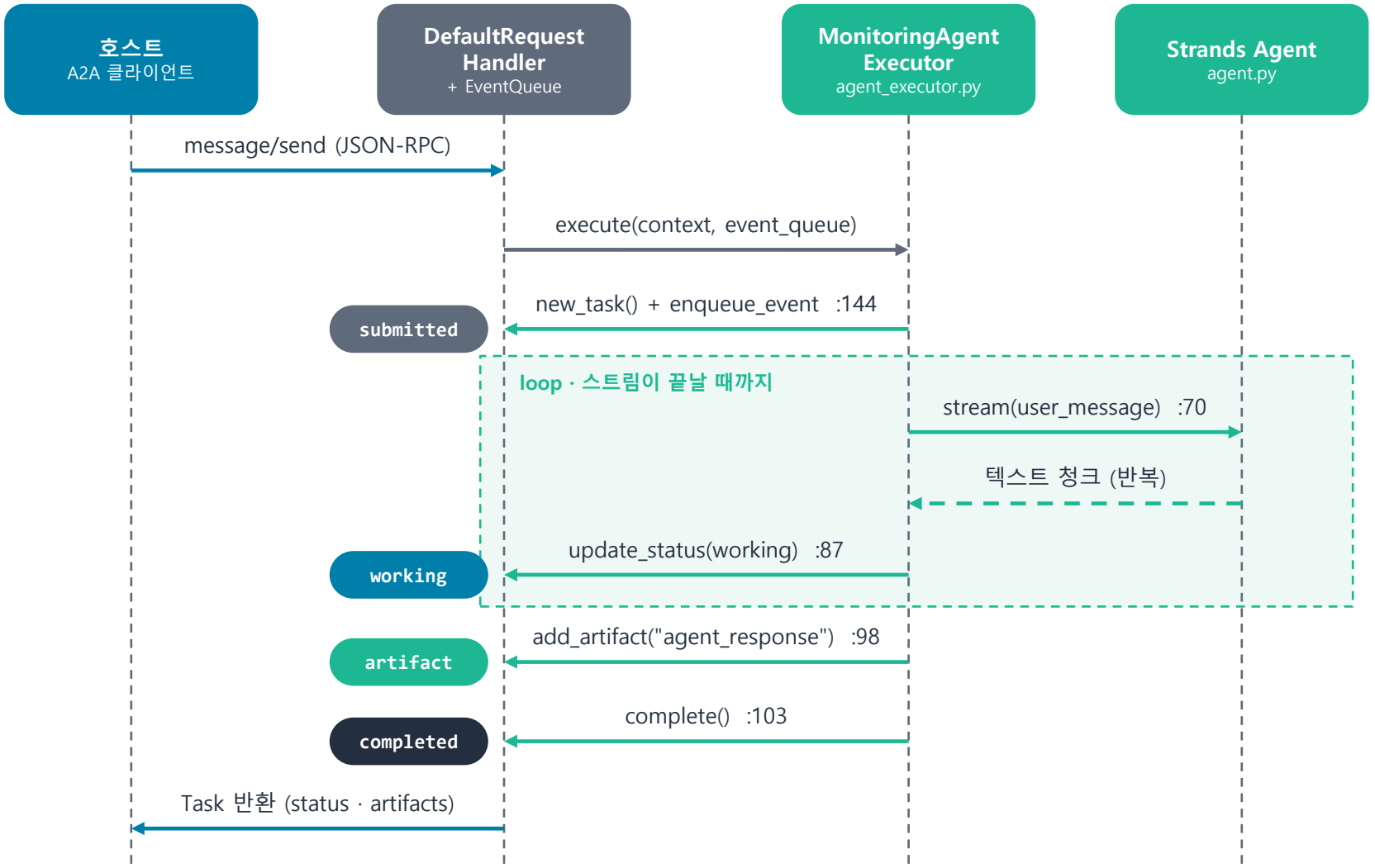
→ **AgentCore Runtime**
JWT 검증 · 경로 전달

→ **컨테이너 :9000**
/.well-known/agent-card.json

→ **카드의 url 필드**
= AGENTCORE_RUNTIME_URL
다음 JSON-RPC 호출 주소

# 태스크와 아티팩트: 요청 하나의 수명

교안 #18




## 최종 결과: Task 객체

```
{
  "kind": "task",
  "id": "…",
  "contextId": "…",
  "status": {
    "state": "completed" },
  "artifacts": [{
    "name": "agent_response",
    "parts": [{ "kind": "text",
                "text": "…" }]
  }]
}
```

**상태 메시지**
진행 중 업데이트 · 매번 누적 전체 텍스트

**아티팩트**
최종 결과물 · 호스트가 쓰는 답 (streaming=False)

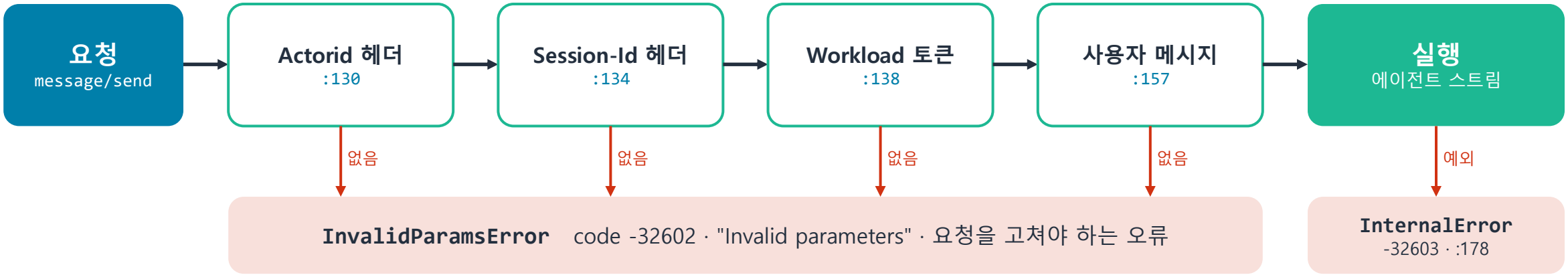
# 인터페이스 계약: 선언 · 검증 · 오류

교안 #24

## ① 선언 에이전트 카드 · main.py:37

| 입력 모드 | 출력 모드 | capabilities | skills | version | securitySchemes |
|---|---|---|---|---|---|
| text/plain | text/plain | streaming ✓ · push ✗ | 5개 · id · description · tags | 0.3.0 | 선언 없음 (실제로는 JWT 필요) |

## ② 검증 실행기 · agent_executor.py:128–157 하나라도 없으면 거부




## ③ 오류 형식 JSON-RPC 2.0 표준 오류 응답

```
{ "jsonrpc": "2.0", "id": 1,
  "error": { "code": -32602, "message": "Invalid parameters" } }
```

**오류도 계약이다**
상대 프레임워크를 몰라도 오류 코드만으로
'내 요청 문제'와 '서버 문제'를 구분

# 보안: 아웃바운드 토큰과 인바운드 검증

교안 #25

## Host 런타임 · 아웃바운드
내가 남을 부를 자격

```
# host_adk_agent/agent.py:40
@requires_access_token(
  provider_name=provider_name,
  scopes=[], auth_flow="M2M",
  into="bearer_token",
  force_authentication=True)
def _create_client(bearer_token=""):
  headers = {"Authorization":
    f"Bearer {bearer_token}", ...}
```

**코드에 시크릿 없음 · 공급자 이름만 SSM에서**
`/monitoragent/agentcore/provider-name`

⑤ A2A 요청 · Authorization: Bearer <token>

① 토큰 요청

④ bearer_token 주입

### AgentCore Identity
OAuth2 자격 증명 공급자
provider_name → client_id + secret 보관

② client_credentials
③ access token

### Amazon Cognito
토큰 엔드포인트 · M2M 토큰 1시간

**CustomJWTAuthorizer**
1 DiscoveryUrl로 서명 키 조회
2 서명 · 만료 검증
3 client_id ∈ AllowedClients

통과

**컨테이너 :9000 · A2A 서버**

**Monitoring 런타임 · 인바운드 — 누가 나를 부를 수 있나**

실패 → 401/403, 컨테이너는 요청을 못 봄

```
# cloudformation/monitoring_agent.yaml – 인바운드 (코드 0줄)
AuthorizerConfiguration:
  CustomJWTAuthorizer:
    DiscoveryUrl: <Cognito DiscoveryUrl>
    AllowedClients: [<MonitoringClientId>]
```

**아웃바운드** Identity가 발급 · 코드에 시크릿 0줄
**인바운드** Runtime이 검증 · 코드 0줄 (설정만)
같은 토큰이 양쪽을 잇는다: monitoring-client 하나로 발급하고 허용

# 메모리: 꼭 짚을 코드 6곳

교안 #30–36

| 교안 | 개념 | 코드 위치 | 짚을 포인트 |
|---|---|---|---|
| **#30** | 단기 / 장기 메모리 | `monitoring_strands_agent/memory_hook.py:118, 59` | create_event()는 단기 이벤트, retrieve_memories()는 장기 레코드 |
| **#31** | 프레임워크 무관 | `memory_hook.py` · `web_search_openai_agents/memory_tool.py` | Strands는 훅, OpenAI Agents SDK는 @function_tool — 같은 Memory API |
| **#32** | 공유 memory_id | `cloudformation/monitoring_agent.yaml:845` | 차이점: 샘플은 에이전트마다 Memory 리소스가 따로 있음 |
| **#33** | 메모리 훅 | `monitoring_strands_agent/memory_hook.py:171-173` | 콜백 3개: 초기화 · 메시지 추가 · 호출 후 |
| **#35–36** | invocation_state | `host_adk_agent/agent.py:48-52` | 런타임이 다르면 HTTP 헤더가 invocation_state 역할을 대신함 |

# B

# 추가적인 기술 내용

샘플이 실제로 동작하게 만드는 Runtime · 인증 · Gateway · 메모리 설계

# Runtime 서비스 계약: HTTP와 A2A

NEW · Runtime

## Host · ProtocolConfiguration: HTTP

호출자: 프런트엔드 (사용자 JWT)

- **포트 8080** POST /invocations · GET /ping
- **BedrockAgentCoreApp** 이 계약을 대신 구현
- **async generator**로 yield하면 그대로 스트리밍 응답

```
# host_adk_agent/main.py
app = BedrockAgentCoreApp()

@app.entrypoint
async def call_agent(payload, context):
    session_id = context.session_id
    ...
    async for event in runner.run_async(...):
        yield event
```

## Remote · ProtocolConfiguration: A2A

호출자: 호스트 에이전트 (M2M JWT)

- **포트 9000 루트(/)** JSON-RPC 2.0
- **A2AStarletteApplication**이 카드 경로를 자동 게시
- **/ping**은 코드에서 직접 추가 (main.py:89)

```
# monitoring_strands_agent/main.py
server = A2AStarletteApplication(
    agent_card=agent_card,
    http_handler=request_handler)
app = server.build()

@app.route("/ping", methods=["GET"])
async def ping(request): ...
uvicorn.run(app, host="0.0.0.0", port=9000)
```

# 에이전트 카드는 어디에 있고 무엇을 담는가

NEW · Discovery

| https://bedrock-agentcore.{region}.amazonaws.com | /runtimes/{URL 인코딩된 ARN} | /invocations | /.well-known/agent-card.json |
|---|---|---|---|
| AgentCore 데이터 플레인 | quote(ARN, safe='') | Runtime 호출 경로 | A2A 표준 카드 경로 |

```
# host_adk_agent/agent.py
MONITOR_AGENT_ID = get_ssm_parameter(
    "/monitoragent/agentcore/runtime-id")
MONITOR_AGENT_ARN = f"arn:aws:bedrock-agentcore:"
    f"{region}:{account_id}:runtime/{MONITOR_AGENT_ID}"

monitor_agent_card_url = (
  f"https://bedrock-agentcore.{region}.amazonaws.com"
  f"/runtimes/{quote(MONITOR_AGENT_ARN, safe='')}"
  "/invocations/.well-known/agent-card.json")
```

- **SSM으로 느슨한 결합:** 호스트는 원격 런타임 ID만 알면 됨 (배포 순서와 무관)
- **카드 조회도 인증 대상:** Runtime의 JWT 검증을 거치므로 인증된 클라이언트로 조회
- **카드의 url:** AGENTCORE_RUNTIME_URL 환경변수, 없으면 127.0.0.1:9000 (로컬 테스트)
- **카드 불일치:** Smithy 모델은 CloudWatch 작업 7개, 카드 skills는 로그 4개 + 검색 도구만 하드코딩

# 컨텍스트 전파: 세 개의 헤더

NEW · 교안 #35

프런트엔드 → Host 런타임 → Monitoring · WebSearch 런타임

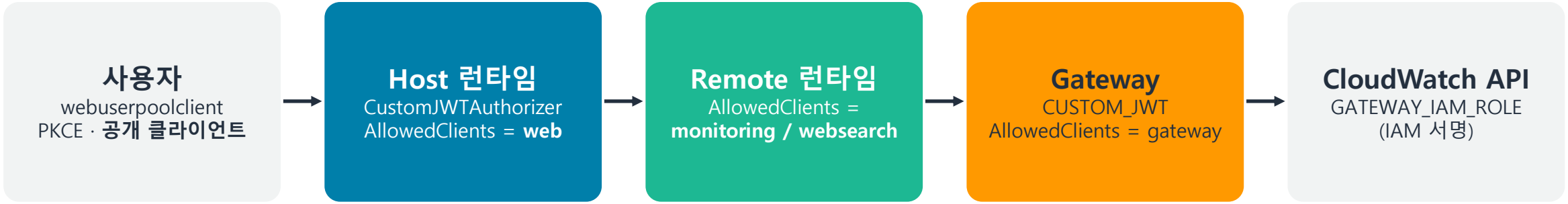
| 헤더 | 종류 | 출처 → 사용처 |
|---|---|---|
| `X-Amzn-Bedrock-AgentCore-Runtime-Session-Id` | Runtime 기본 | 프런트엔드 생성 → 호스트 context.session_id → 하위로 그대로 전달 → microVM 라우팅 · 메모리 session_id |
| `X-Amzn-Bedrock-AgentCore-Runtime-Custom-Actorid` | 사용자 정의 (allowlist 필수) | 프런트엔드가 JWT sub를 넣음 → 호스트 request_headers → 하위 전달 → 메모리 actor_id |
| `X-Amzn-Bedrock-AgentCore-Runtime-Workload-AccessToken` | Runtime이 주입 | 인바운드 JWT 검증 후 Runtime이 추가 → 모니터링 에이전트가 Gateway 토큰 교환에 사용 |

```
# cloudformation/monitoring_agent.yaml
RequestHeaderConfiguration:
  RequestHeaderAllowlist:
    - X-Amzn-Bedrock-AgentCore-Runtime-Custom-Actorid
```

**교안 #35와 비교**
같은 프로세스: invocation_state 딕셔너리
다른 런타임: HTTP 헤더 + 런타임 allowlist

# 인증 체인: 3번의 홉, 4개의 Cognito 클라이언트

NEW · 교안 #25




```python
# host_adk_agent/agent.py — 시크릿 없이 토큰 주입
@requires_access_token(
    provider_name=provider_name,  # SSM에서 읽은 이름
    scopes=[], auth_flow="M2M",
    into="bearer_token", force_authentication=True)
def _create_client(bearer_token: str = str()):
    headers = {"Authorization": f"Bearer {bearer_token}", ...}

# monitoring_strands_agent/utils.py — 두 번째 홉
agentcore_client.get_resource_oauth2_token(
    workloadIdentityToken=workload_token,
    resourceCredentialProviderName=GATEWAY_PROVIDER_NAME,
    oauth2Flow="M2M")
```

- **홉마다 다른 audience:** 토큰 하나가 유출돼도 다른 홉을 통과하지 못함
- **시크릿은 Identity가 보관:** 커스텀 리소스가 OAuth2 자격 증명 공급자를 생성, 코드는 이름만 앎
- **M2M 토큰 수명 1시간:** cognito.yaml AccessTokenValidity
- **대비:** Tavily API 키는 Runtime 환경변수에 평문 (web_search_agent.yaml:922)

# ADK sub_agents는 '도구 호출'이 아니라 '제어 이양'

NEW · 교안 #17, #19

### sub_agents (현재 샘플)
제어 이양 · transfer

- LLM이 transfer_to_agent로 대상 선택
- 하위 에이전트의 출력이 곧 응답
- RemoteA2aAgent는 부모로 되돌아가지 못함

```
sub_agents=[monitor_agent,
            websearch_agent]
```

### AgentTool
도구로서의 에이전트 · 교안 #17

- 하위 에이전트를 도구로 호출
- 결과가 부모에게 돌아와 합성
- 부모가 대화 제어권 유지

```
tools=[AgentTool(agent=
        monitor_agent)]
```

### SequentialAgent / Graph
결정적 순서 · 교안 #19–21

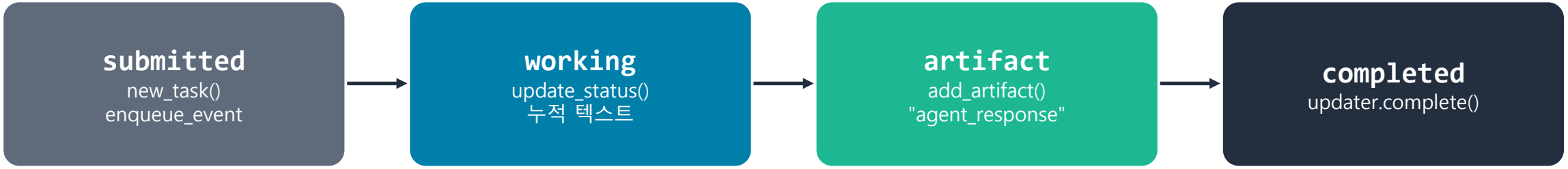
- 순서를 코드로 보장
- monitor → websearch → 합성
- LLM 판단에 맡기지 않음

```
SequentialAgent(sub_agents=
  [monitor, websearch, synth])
```

**확인 포인트** 호스트 프롬프트의 "First monitor → Then websearch → Finally synthesize"는 지시일 뿐입니다. 한 턴에 두 에이전트가 모두 호출되는지 데모에서 직접 관찰하세요.

# AgentExecutor: 프레임워크 스트림 → A2A 태스크

NEW · 교안 #18




`cancel() → _active_tasks[task_id] = False → 스트림 루프가 확인하고 중단 → canceled`

**Strands (모니터링)**

```
async for event in self.agent.stream_async(query):
    if "data" in event:
        yield {"content": event["data"], ...}
```

**OpenAI Agents SDK (웹 검색)**

```
result = Runner.run_streamed(agent, input=prompt)
async for event in result.stream_events():
    if event.type == "raw_response_event" and \
       isinstance(event.data, ResponseTextDeltaEvent):
```

- **다른 것은 이벤트를 읽는 한 줄뿐:** 실행기 구조는 두 에이전트가 거의 같음
- **오류 매핑:** 헤더 누락 → InvalidParamsError, 그 외 예외 → InternalError (JSON-RPC 오류)
- **누적 전송과 휘발성:** 업데이트마다 전체 텍스트를 다시 보냄 · InMemoryTaskStore는 재시작 시 소멸

# 메모리 전략 설계: 네임스페이스가 곧 스키마

NEW · 교안 #30

| 전략 | Monitoring 에이전트 | Web Search 에이전트 |
|---|---|---|
| **Custom (프롬프트 덮어쓰기)** | MonitoringIssueTracker → /technical-issues/{actorId} | SearchQueryTracker → /search-queries/{actorId} |
| **Semantic** | /knowledge/{actorId} | /knowledge/{actorId} |
| **Summary** | — | SessionSummarizer → /summaries/{actorId}/{sessionId} |
| **UserPreference** | /users/{actorId} (조회 코드 없음) | /users/{actorId} |

```
# cloudformation/monitoring_agent.yaml
CustomMemoryStrategy:
  Name: MonitoringIssueTracker
  Namespaces: ["/technical-issues/{actorId}"]
  Configuration:
    SemanticOverride:
      Extraction:    { AppendToPrompt: ..., ModelId: ... }
      Consolidation: { AppendToPrompt: ..., ModelId: ... }
EventExpiryDuration: 60    # 단기 이벤트 60일
```

- **추출 지시:** 지표 값 · 단위 · 타임스탬프 · 리소스 ID만 사실로 추출, 추측 금지
- **통합 지시:** 사고 타임라인 보존, 계정 · 리전 · 환경 구분 유지
- **조회 튜닝:** technical-issues top_k 3 · 점수 ≥ 0.3 / knowledge top_k 5 · 점수 ≥ 0.2
- **MemoryExecutionRole:** 커스텀 전략이 직접 Bedrock 모델을 호출하므로 필요

# 메모리 통합: 훅(결정적)과 도구(LLM 판단)

NEW · 교안 #31, #33

## Strands 훅 — monitoring_strands_agent/memory_hook.py

1. **AgentInitializedEvent**
   `get_last_k_turns(k=5) → <recent-conversation>`
2. **MessageAddedEvent**
   `retrieve_memories() → <memory-context>`
3. **AfterInvocationEvent**
   `create_event(USER, ASSISTANT)`

항상 같은 동작 · 에이전트는 메모리를 모름
주의: 시스템 프롬프트에 계속 덧붙이기만 함 (:78, :152)

## @function_tool — web_search_openai_agents/memory_tool.py

```
retrieve_monitoring_context(query, context_type)
save_interaction_to_memory(user_msg, response)
get_recent_conversation_history(k_turns=5)
save_custom_memory(content, memory_type)
search_memory_by_namespace(query, namespace_type)

# 네임스페이스는 실행 시 조회
strategies = client.get_memory_strategies(memory_id)
```

유연하지만 LLM이 호출해야만 동작
프롬프트의 '저장은 자동 처리됨'과 실제 동작이 다름

# 배포와 운영 자동화

NEW · 교안 #10, #38

**Step 0**
S3 + Smithy 모델 업로드

→

**Step 1**
Cognito 스택
클라이언트 4개

→

**Step 2–4**
에이전트 스택 3개
(병렬 가능)

deploy.py · 배포 순서

**SSM으로 느슨한 결합**

```
/monitoragent/agentcore/runtime-id
/websearchagent/agentcore/provider-name
호스트는 실행 시점에 읽음 → 스택 순서 무관
```

**CodeBuild**
ARM 컨테이너
build-N 태그

→

**ECR**
PUSH SUCCESS

→

**EventBridge**
ECR Image Action

→

**Lambda**
update_agent_
runtime()

→

**Runtime**
새 버전

이미지 갱신 루프 · 에이전트 스택마다 하나씩

- **allowlist 재지정:** update_agent_runtime 호출 시 requestHeaderAllowlist를 다시 넘김 — 빠뜨리면 Actorid 헤더 소실
- **관찰성:** CMD ["opentelemetry-instrument", ...] + aws-opentelemetry-distro, Logs Delivery, X-Ray. OpenAI SDK는 OpenAIAgentsInstrumentor 추가
- **버전 고정:** 프런트엔드는 qualifier=DEFAULT 엔드포인트 호출 — 카나리 배포는 별도 엔드포인트 필요

# 코드에서 찾은 함정: 토론 · 실습 과제

교안 #6, #12, #25

## 1 actor ID 위장

교안 #25

actor ID를 검증된 JWT가 아니라 요청 헤더에서 읽음 → 헤더만 바꾸면 다른 사용자의 메모리에 접근

`host_adk_agent/main.py:31`

## 2 시스템 프롬프트 누적

교안 #6

메모리를 매 메시지마다 시스템 프롬프트 끝에 덧붙이고 지우지 않음

`memory_hook.py:78, 152`

## 3 프롬프트 불일치

교안 #24

프롬프트가 존재하지 않는 도구(list_cloudwatch_dashboards 등)를 지시, {question} 자리표시자도 채워지지 않음

`monitoring_strands_agent/prompt`

## 4 카드 불일치

교안 #24

카드 skills를 하드코딩해서 Metrics · Alarms 작업이 빠짐

`monitoring_strands_agent/main.py:37`

## 5 복원력 부재

교안 #12

A2A 호출에 재시도 · 회로 차단기 · 부분 응답이 없음. 하위 실패는 InternalError로 전파

`host_adk_agent/agent.py`

## 6 작은 버그

교안 —

raise f"..." 는 TypeError 발생, /users 네임스페이스는 추출만 하고 조회하지 않음

`monitoring_strands_agent/agent.py:75`

정리

# 교안의 개념은 코드 한 줄로 내려온다

## 통신

ProtocolConfiguration: A2A 한 줄로 서버가 되고, 카드 URL은 인코딩한 ARN과 /.well-known 경로로 만든다

## 보안

홉마다 다른 클라이언트와 토큰을 쓰고, 시크릿은 AgentCore Identity가 보관한다

## 상태

세션은 microVM이 격리하고, 컨텍스트는 헤더로 전파하며, 장기 기억은 전략별 네임스페이스에 남는다